

Machine Learning - Capstone Project

Guidelines, Algorithm List & Evaluation Rubrics

B.Tech. (Computer Science and Engineering) - III Year — 23CSE301 Machine Learning
Academic Year 2026-27

1 Overview

The capstone project requires each team to build a **complete, end-to-end Machine Learning pipeline** for three problem tracks: **Regression, Classification, and Clustering**. The pipeline covers the full workflow, data loading, exploratory data analysis (EDA), cleaning and feature engineering, model training and comparison, hyperparameter tuning, and result visualisation, using the dataset and problem statement assigned by the instructor.

Team size	3 members
Tracks	Regression, Classification, Clustering
Reviews	Two formal reviews; 25 marks each (total 50 marks)
Datasets	Assigned by instructor (one dataset per track per team)
Primary tools	Python 3, scikit-learn, Pandas, NumPy, Matplotlib, Seaborn
Submission	Jupyter Notebook(s) + GitHub repository

2 Review Schedule and Track Scope

Review	Date	Scope
Review 1	Just before mid-semester examinations	Full Regression track + Classification Part A
Review 2	Towards the end of the semester	Classification Part B + Full Clustering track

Both reviews are conducted as in-person presentations in front of the instructor. All team members are expected to be present and must be able to answer questions on any part of the submitted work.

3 Algorithm Requirements by Track

3.1 Regression Track (Evaluated in Review 1)

All ten algorithms listed below must be trained on the *same* preprocessed dataset and evaluated on the *same* held-out test set.

#	Algorithm	Notes
1	Linear Regression	Baseline; interpret coefficients
2	Ridge Regression	L_2 regularisation; tune <code>alpha</code>
3	Lasso Regression	L_1 regularisation; observe feature sparsity
4	ElasticNet Regression	Combined L_1+L_2 ; tune <code>l1_ratio</code>
5	Polynomial Regression	Apply <code>PolynomialFeatures</code> then Linear Regression; compare degrees
6	Decision Tree Regressor	Tune <code>max_depth</code> ; show feature importance
7	Random Forest Regressor	Ensemble baseline; tune <code>n_estimators</code>
8	Gradient Boosting Regressor	<code>sklearn</code> GBM <i>or</i> XGBoost; tune <code>learning_rate</code>
9	Support Vector Regressor (SVR)	Scale features first; tune <code>C</code> and <code>kernel</code>
10	K-Nearest Neighbors Regressor	Tune <code>k</code> ; discuss impact of scaling

Evaluation metrics (all mandatory): R^2 score, RMSE, MAE, and 5-fold cross-validated R^2 for the two best-performing models.

3.2 Classification Track

Part A is evaluated in Review 1; **Part B** in Review 2. The final comparison table in Review 2 must include all ten algorithms.

#	Algorithm	Notes
1	Logistic Regression	Baseline classifier; interpret coefficients/odds
2	K-Nearest Neighbors	Tune <code>k</code> ; discuss distance metrics
3	A Naive Bayes (Gaussian)	Discuss conditional independence assumption
4	Decision Tree Classifier	Tune <code>max_depth</code> ; visualise the tree
5	Support Vector Machine (SVC)	Tune <code>C</code> and <code>kernel</code> ; scale features
6	Random Forest Classifier	Feature importance; tune <code>n_estimators</code>
7	AdaBoost Classifier	Tune <code>n_estimators</code> and <code>learning_rate</code>
8	B Gradient Boosting Classifier	<code>sklearn</code> GBM <i>or</i> XGBoost/LightGBM
9	Bagging Classifier	Use Decision Tree as base estimator
10	MLP Classifier (Neural Network)	Tune <code>hidden_layer_sizes</code> and <code>activation</code>

Evaluation metrics (all mandatory): Accuracy, Precision, Recall, F1-score (weighted), Confusion matrix, and ROC-AUC score. For multi-class problems use One-vs-Rest (OvR) for ROC-AUC. The final Review 2 deliverable must include a single consolidated comparison table across all ten algorithms.

3.3 Clustering Track (Evaluated in Review 2)

Unlike regression and classification, clustering has no labels during training. Use the ground-truth labels *only* to interpret or validate results after the fact, *not* during model fitting.

#	Algorithm	Notes
1	K-Means Clustering	Plot Elbow curve (inertia vs. k) to choose k
2	Agglomerative Hierarchical Clustering	Plot Dendrogram; compare linkage strategies

Evaluation metrics (all mandatory): Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index for all algorithms. Cluster visualisation via **PCA** (2 principal components) is mandatory; t-SNE visualisation is encouraged.

4 Deliverables per Review

#	Deliverable	Review 1	Review 2
D1	Jupyter Notebook(s) — clean, fully run, outputs visible	✓	✓
D2	GitHub repository with meaningful commit history	✓	✓
D3	Comparative results table (algorithms vs. metrics)	✓	✓
D4	All required visualisations (see rubric)	✓	✓
D5	README file (dataset description, instructions to run)	—	✓
D6	Consolidated 10-algorithm classification comparison	—	✓
D7	Cluster visualisations (Elbow, Dendrogram, PCA plot)	—	✓
D8	GUI / Deployed application (bonus)	—	optional

5 Review 1 Rubric - 25 Marks

#	Criterion	Full-marks descriptor	Max
Section A - Dataset & EDA			4
A1	Dataset loading & audit	Shape, dtypes, missing-value counts, and class/target distribution all reported clearly	1
A2	EDA visualisations	At minimum: distribution plots for each feature, correlation heatmap, target distribution, and at least two scatter plots showing feature-target relationships	2
A3	Insight commentary	Each major visualisation is accompanied by a written observation (Markdown cell) noting what the plot reveals about the data	1
Section B - Preprocessing & Feature Engineering			3
B1	Data cleaning	Missing values handled with a justified strategy; duplicates and outliers checked and treated	1
B2	Encoding, scaling & splitting	Appropriate encoding for categoricals; correct scaler applied (fitted on train set only); stratified train/test split used	1
B3	Feature engineering	At least one engineered feature created with a written justification of why it may improve model performance	1
Section C - Regression Track			9
C1	Implementation	All 10 required regression algorithms trained, predictions generated, no errors in notebook output	4
C2	Comparative evaluation	Single summary table showing R^2 , RMSE, and MAE for all 10 models on the same test split; models ranked by R^2	2
C3	Hyperparameter tuning	<code>GridSearchCV</code> or <code>RandomizedSearchCV</code> applied to at least 2 models; best parameters and improvement in metric reported	2
C4	Visualisation	Residual plot and predicted-vs-actual plot for the best model; feature importance plot for at least one tree-based model	1
Section D - Classification Track, Part A			3
D1	Implementation	All 5 Part-A algorithms trained and producing predictions with no errors	2
D2	Evaluation	Accuracy, weighted F1, and confusion matrix reported per algorithm; preliminary comparison table presented	1
Section E - Presentation			1
E1	Presentation quality	Clear narrative linking each step; all team members can explain their code	1
Viva			5
Total			25

6 Review 2 Rubric - 25 Marks + up to 2 Bonus

#	Criterion	Full-marks descriptor	Max
Section A - Classification Track, Part B			6
A1	Implementation	All 5 Part-B algorithms trained on the same dataset as Review 1 Part A, no errors	3
A2	Full comparison table	A single table comparing all 10 classification algorithms on Accuracy, Precision, Recall, F1, and ROC-AUC	1
A3	Final model selection	Best model identified with clear justification; hyperparameter tuning applied; improvement in at least one metric documented	2
Section B - Clustering Track			8
B1	Implementation	All 2 required clustering algorithms fitted and cluster labels obtained; no errors	4
B2	Evaluation metrics	Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index reported for all algorithms; Elbow curve for K-Means and Dendrogram for Hierarchical clustering shown	2
B3	Cluster visualisation	PCA-reduced 2D scatter plot with cluster colour-coding for every algorithm; t-SNE plot for at least one algorithm	2
Section C - Pipeline Integration & Quality			3
C1	Code quality	Notebooks are clean (cells run top-to-bottom without errors), functions are modular, and key steps are commented	1
C2	README & documentation	README.md includes: dataset description, problem statement, results summary table, environment setup, and how-to-run instructions	1
C3	GitHub repository	Repository has a meaningful commit history (not a single bulk upload); <code>requirements.txt</code> or <code>environment.yml</code> present	1
Section D - Presentation & Viva			8
D1	Presentation	Story arc: problem → data → method → results → conclusion; visualisations are clear; slides/notebook is well-organised	3
D2	Viva	All team members demonstrate understanding; questions on algorithm choice, metric interpretation, and trade-offs are answered	5
Total			25

Bonus Opportunities (up to +2 marks in Review 2)

+1	Interactive GUI	A working web-based interface (Streamlit or Gradio recommended) that accepts user inputs and returns predictions from the trained model.
+1	Public Deployment	The application is deployed and accessible via a public URL (e.g. Streamlit Community Cloud, Hugging Face Spaces, or Render).
+2	Both	Both a working GUI <i>and</i> a live public deployment.

Bonus marks are added to the Review 2 score but the total displayed is capped at 20 for official grading purposes unless otherwise communicated by the instructor.

7 General Guidelines

7.1 Code and Notebooks

- Use **one notebook per track** (e.g. `regression.ipynb`, `classification.ipynb`, `clustering.ipynb`) or a single notebook with clearly labelled sections.
- All cells must be run **top-to-bottom** before submission with outputs visible. Do not submit notebooks with execution errors.
- Use **Markdown cells** to explain the purpose of each major code block. Do not write a wall of code without explanation.
- **Set a random seed** (`random_state=42`) wherever applicable so results are reproducible.
- All scalers and encoders must be **fitted on the training set only** and applied (transformed) to both the training and test sets. Fitting on the full dataset is a data-leakage error and will result in mark deductions.

7.2 Model Evaluation

- Use a consistent **train/test split** (typically 80:20) across all algorithms within a track so that metric comparisons are fair.
- **Cross-validation** must be used for at least the two best-performing models in each track.
- Present all results in a **summary table** (Pandas DataFrame displayed in the notebook or a formatted Markdown table) – not as scattered individual print statements.

7.3 Visualisations

- Every plot must have a **title**, labelled axes, and a **legend** where applicable.
- Use `plt.tight_layout()` or `fig.savefig(..., bbox_inches='tight')` to prevent clipped labels.
- Colourblind-friendly palettes (`'tab10'`, `'Set2'`, or Seaborn's `'colorblind'`) are strongly recommended.

7.4 Team Responsibilities

- Each team member should be the primary owner of *at least one* track. Ownership means they can explain every line of code in that section during the viva.
- Split work is expected; **divide by track, not by step**. Avoid splitting “one person does EDA, one does models, one does slides” – each person should understand the full pipeline for their assigned track.
- During the viva, the teacher may direct questions to *any* team member about *any* part of the work.

7.5 Academic Integrity

- Code taken from external sources (StackOverflow, blogs, etc.) must be cited in a Markdown comment.

- Each team's analysis, interpretation, and feature engineering decisions must be **original**. Identical notebooks across teams will result in zero for both teams.
- Generative AI tools may be used for code scaffolding but *not* for analysis or interpretation. If AI assistance is used, cite it in the README.

8 GitHub Repository Structure

The repository must follow this directory structure at minimum:

/	(root)
README.md	Project overview, results, setup instructions
requirements.txt	All Python dependencies with versions
data/	Raw dataset file(s) <i>or</i> a script to download them
notebooks/	regression.ipynb, classification.ipynb, clustering.ipynb
models/	Saved model files (optional; .pkl via joblib)
app/	GUI / deployment code (if attempting bonus)

Commit at every meaningful milestone (e.g. "Add Random Forest with GridSearchCV", "Complete clustering EDA"). A single commit containing all work will receive zero marks for the GitHub criterion.

9 Mark Summary

Component	Review 1	Review 2
Dataset & EDA	4	—
Preprocessing & Feature Engineering	3	—
Regression Track	9	—
Classification Track Part A	3	—
Classification Track Part B	—	6
Clustering Track	—	8
Pipeline Integration & Code Quality	—	3
Presentation & Viva	6	8
Subtotal	25	25
Bonus (GUI + Deployment)	—	up to +2
Grand Total	50 marks (+ up to 2 bonus)	